

AntiFLipper: A Secure and Efficient Defense Against Label-Flipping Attacks in Federated Learning

Aashnan Rahman¹, Abid Hasan², Sherajul Arifin³, Faisal Haque Bappy⁴, Tahrim Hossain⁵,
Tariqul Islam⁶, Abu Raihan Mostofa Kamal⁷, and Md. Azam Hossain⁸
^{1,2,3,7,8} Islamic University of Technology, Bangladesh.
^{4,5,6} Syracuse University, United States.

Email: {aashnanrahman@iut-dhaka, abidhasan32@iut-dhaka, sherajularifin@iut-dhaka,
fbappy@syr, mhossa22@syr, mtislam@syr, azam@iut-dhaka, raihan.kamal@iut-dhaka}.edu

Abstract—Federated learning (FL) enables privacy-preserving model training by keeping data decentralized. However, it remains vulnerable to label-flipping attacks, where malicious clients manipulate labels to poison the global model. Despite their simplicity, these attacks can severely degrade model performance, and defending against them remains challenging. We introduce AntiFLipper, a novel and computationally efficient defense against multi-class label-flipping attacks in FL. Unlike existing methods that ensure security at the cost of high computational overhead, AntiFLipper employs a novel client-side detection strategy, significantly reducing the central server’s burden during aggregation. Comprehensive empirical evaluations across multiple datasets under different distributions demonstrate that AntiFLipper achieves accuracy comparable to state-of-the-art defenses while requiring substantially fewer computational resources in server side. By balancing security and efficiency, AntiFLipper addresses a critical gap in existing defenses, making it particularly suitable for resource-constrained FL deployments where both model integrity and operational efficiency are essential.

Index Terms—Federated Learning, Label Flipping Attack, Learning, Malicious Node Detection, Attack Mitigation

I. INTRODUCTION

Federated Learning (FL) is a distributed machine learning framework that enables multiple clients to collaboratively train a global model while keeping their data localized, thereby preserving privacy and addressing critical data security concerns [1], [2]. By aggregating locally trained model updates rather than raw data, FL has gained traction in domains such as healthcare, finance, and mobile computing where data privacy is paramount. However, the decentralized nature of FL also exposes it to a range of adversarial threats [3], [4]. One of the most potent among these is the label-flipping attack, where malicious clients intentionally mislabel their local data to poison the training process [5], [6]. These corrupted updates, when combined with honest ones, can severely degrade the global model’s accuracy and reliability [7].

The simplicity of label-flipping attacks belies their potency. Unlike complex poisoning schemes requiring sophisticated adversarial techniques or model manipulation, label-flipping merely requires access to label modification capabilities [8]. A malicious client can implement such attacks with minimal technical expertise, making them particularly concerning in real-world FL deployments. As demonstrated by Tolpegin et al. [9], even a small fraction of compromised clients can

cause substantial degradation in global model accuracy through targeted label-flipping.

Existing defense mechanisms against label-flipping attacks typically face a challenging trade-off between security effectiveness and computational efficiency. Lightweight approaches may work well for simple datasets but often fail to detect targeted attacks in complex data distributions, particularly in non-IID (non-Independent and Identically Distributed) settings common in real-world FL deployments [10]. Conversely, more effective defenses achieve higher accuracy but at the cost of significantly increased aggregation time, making them impractical for resource-constrained environments or time-sensitive applications [11].

To address these limitations, we introduce AntiFLipper, a novel defense mechanism designed to detect and mitigate label-flipping attacks in federated learning environments with both high accuracy and low computational overhead. Unlike existing approaches that sacrifice efficiency for security or vice versa, AntiFLipper achieves an optimal balance by employing a specialized detection algorithm that identifies potentially malicious updates without requiring extensive computational resources during the aggregation phase.

The following are the core contributions of this paper,

- **Attack Detection & Mitigation:** We propose AntiFLipper, a novel trust-based weighted aggregation method that detects and mitigates multi-class label-flipping attacks in federated learning by identifying and eliminating malicious clients both in standard and dynamic scenarios.
- **Accuracy with Reduced Aggregation Time:** AntiFLipper matches the performance of state-of-the-art defenses in terms of accuracy, while significantly reducing aggregation time.
- **Minimizing Server Dependency for Better Scalability:** By shifting computational responsibilities to client devices, AntiFLipper minimizes reliance on the central server, improving scalability and efficiency in decentralized learning settings.

The remainder of the paper is structured as follows. In Section II, we describe the problem statement and our research objectives. In Section III, we review existing work related to the detection and defense against label-flipping attacks by malicious clients in federated learning scenarios. In Section IV,

we present the system architecture of AntiFLipper. In Section V we present system implementation and performance evaluation. In Section VI, we provide a security analysis, discussing the robustness of our approach in mitigating malicious attacks. Finally, Section VIII summarizes the key findings and concludes the paper.

II. PROBLEM STATEMENT

A. Label Flipping Attack

Let $C = \{c_1, c_2, \dots, c_n\}$ be the set of clients participating in the federated learning process. Each client $c_i \in C$ possesses a local dataset $D_i = \{(x_k, y_k)\}_{k=1}^{m_i}$, where $x_k \in \mathcal{X}$ is the input feature and $y_k \in \mathcal{Y}$ is the true class label. In a label-flipping attack, the adversary modifies this dataset into a corrupted version $D'_i = \{(x_k, y'_k)\}_{k=1}^{m_i}$, such that $y'_k \neq y_k$ for a substantial subset of the data [12]. These poisoned labels cause the local model m'_i , trained on D'_i , to encode incorrect decision boundaries. A common variant is the *targeted label-flipping attack*, where the adversary selects a source class $s \in \mathcal{Y}$ and a target class $\tau \in \mathcal{Y}$, applying the transformation:

$$y'_k = \begin{cases} \tau & \text{if } y_k = s, \\ y_k & \text{otherwise} \end{cases}$$

The malicious client then trains locally and submits the resulting model m'_i to the server, which aggregates updates at each training round i as M_{i+1} . Over time, the aggregation of poisoned updates subtly shifts the global model's decision boundaries toward the adversary's target class [13] without requiring server access, disrupting communication, or deviating from the standard protocol, relying solely on local label manipulation.

B. Research Objectives

Existing techniques to combat label-flipping attacks often rely on centralized aggregation and anomaly detection at the server, assuming full visibility into client behavior and trust in their honesty—assumptions that rarely hold in practice. This dependency on a trusted server introduces fundamental challenges where data sensitivity, model integrity, and system resilience are critical. The centralized design not only creates a single point of failure but also contradicts the privacy-preserving principles of FL. Furthermore, centralized defenses typically incur significant computation and communication overhead, limiting scalability and real-world applicability.

To address these challenges, our research pursues the following three core objectives: i) *design a decentralized aggregation method that accurately detects and mitigates label-flipping attacks*; ii) *maintain high model performance* and reduce aggregation time even in the presence of adversarial clients, ensuring both robustness and efficiency; and iii) *shift the majority of computations to the client side* and rigorously analyze the trade-offs in system efficiency, scalability, and resilience under adversarial conditions. Together, these objectives aim to build a trustworthy federated learning framework that resists adversarial manipulation while preserving client privacy and enabling scalable deployment.

III. RELATED WORK

A. Label Flipping-Specific Defense Mechanisms

Several approaches have tackled label-flipping attacks in federated learning. Tolpegin et al. [9] demonstrated the feasibility of such attacks under varied conditions and proposed a PCA-based mechanism to detect malicious clients. However, their method lacks generalization guarantees and performs poorly in non-IID settings. Fung et al. [6] introduced FoolsGold, which leverages cosine similarity between gradient updates to reduce the influence of coordinated attackers. While effective against sybil-style poisoning, it suffers from false positives in non-IID scenarios and incurs high computational cost due to iterative similarity checks.

Nguyen et al. [11] proposed FLAME, a multi-step defense combining HDBSCAN clustering, differential privacy, and noise addition. It preserves model utility under various backdoor attacks, including label-flipping, but requires careful parameter tuning and introduces substantial overhead. More recently, Jebreel et al. [14] developed LFighter, which reweights client updates using feature-level centroid deviation. Although it improves robustness and offers analytical justification for selective aggregation, its effectiveness declines with data heterogeneity and computational complexity remains a concern.

B. General Federated Learning Defenses Evaluated Against Label Flipping

While not specifically tailored to counter label-flipping attacks, several robust aggregation methods have been studied in this context. FedAvg[1], the standard aggregation mechanism in federated learning, lacks robustness against adversarial updates and suffers significant accuracy degradation under targeted label-flipping. To improve resilience, Yin et al.[10] introduced coordinate-wise *Median and Trimmed Mean* strategies. These methods demonstrate moderate robustness in IID settings but exhibit degraded performance in non-IID scenarios, as they may discard informative but statistically divergent updates, even in the absence of attacks. Multi-Krum [15] extends robustness by using distance-based filtering to eliminate outliers, assuming an IID setting and a known number of adversaries. While effective under these assumptions, its performance deteriorates rapidly with data heterogeneity, and its quadratic computational complexity poses scalability concerns for large-scale systems.

In contrast, our proposed method, AntiFLipper, is designed without relying on data distribution assumptions or prior adversary knowledge. It provides a principled aggregation framework that generalizes well across non-IID environments and varying model architectures. Notably, AntiFLipper significantly reduces computational overhead while maintaining high model accuracy across all evaluated scenarios, making it a practical and scalable solution for real-world federated learning applications where robustness and efficiency are critical.

IV. SYSTEM ARCHITECTURE

The AntiFLipper framework builds on the traditional federated learning paradigm by integrating five interdependent yet modular components as shown in Figure 1. Each module

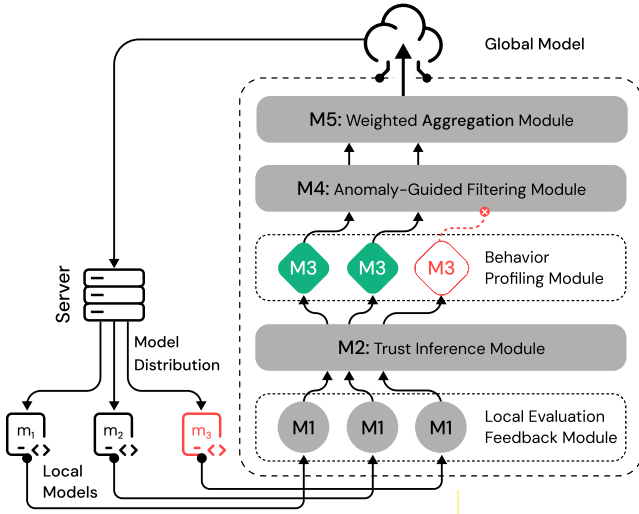


Fig. 1: System Workflow of AntiFLipper

operates in conjunction with the standard FL loop, enhancing robustness against label-flipping attacks without altering the fundamental decentralized design. These modules work together to ensure model integrity, mitigate poisoning, and enforce fairness through dynamic trust-based selection.

A. M1: Local Evaluation Feedback Module

At the beginning of each communication round, the server distributes the global model M_i to all participating clients that are not in the malicious clients list β (Algorithm 1, lines 2–3). Upon receiving the model, each client evaluates M_i on its own local training dataset $data_{ij}$, computing a local accuracy score acc_{ij} . This evaluation step is a lightweight mechanism for identifying adversarial discrepancies. The rationale is based on the assumption that the majority of clients are honest and the global model, trained on clean data, will perform reasonably well on data from similar distributions. Conversely, if a client's data has been poisoned (e.g., via label-flipping), the performance of the global model on this data is expected to degrade sharply. This degradation becomes a signal for potential adversarial behavior. Following evaluation, each client performs local training (Algorithm 1, line 5) to generate a personalized model m_{ij} and returns both the trained model and the evaluation accuracy acc_{ij} to the server.

B. M2: Trust Inference Module

Upon receiving the local models and evaluation results, the server calculates the average accuracy across all non-malicious clients (Algorithm 2, line 1). The trust score τ_j for each client j is then updated according to the deviation from this average. The update rule considers the squared deviation d (Algorithm 2, line 3) between the client's accuracy and the average accuracy: $\tau_j \leftarrow \tau_j \pm \alpha \cdot (acc_{avg} - acc_{ij})^2$. Here, $sqDeviation()$ is defined as $(acc_{ij} - acc_{avg})^2$, while $penalizeTrust()$ and $rewardTrust()$ correspond to $\tau_j - \alpha \cdot d$ and $\tau_j + \alpha \cdot d$, respectively. $normalizeTrust()$ rescales all trust values into $[0, 1]$ excluding malicious clients. The sign is negative if $acc_{ij} < acc_{avg}$ (Algorithm 2, lines 4–5)

Algorithm 1: AntiFLipper()

Input : Number of clients n , client list $clients$, local datasets $data$, learning rate α , trust threshold $\tau_{threshold}$, flag threshold cnt_{max}

Output: Global model M_{i+1} , malicious clients list β

```

1  $(M_0, \tau, cnt, \beta) \leftarrow \text{Initialize}()$ 
2 for  $i \leftarrow 1$  to  $totalRounds$  do
3    $\text{DistributeModel}(M_i, clients, \beta)$ 
4   foreach  $j \in clients$  and  $j \notin \beta$  do
5      $(m_{ij}, acc_{ij}) \leftarrow \text{LocTrain}(M_i, j, data_{ij})$ 
6      $(\tau, cnt, \beta) \leftarrow$ 
        $\text{MalNodeFilter}(clients, acc_{ij}, \tau, cnt, \beta)$ 
7      $M_{i+1} \leftarrow \text{GlobModelAgg}(m, \tau, clients, \beta)$ 
8 return  $M_{i+1}, \beta$ 

```

and positive otherwise (Algorithm 2, lines 6–7). This quadratic penalty function ensures that clients with performance below the mean are penalized more aggressively, while better-than-average performance is rewarded.

This rule resembles gradient descent with a squared loss, where the deviation from the average acts as a proxy for error.

C. M3: Behavior Profiling Module

In this module, the server analyzes the collected accuracy scores and trust values from the clients over multiple rounds to create a behavior profile for each client. This is implemented through the flag counter cnt_j (Algorithm 2, lines 8–11), which tracks how many rounds a client's trust score has fallen below the threshold. This approach enables detection of clients exhibiting consistent anomalies or fluctuations in performance, even if they do not trigger immediate exclusion based on single-round performance.

D. M4: Anomaly-Guided Filtering Module

To avoid overreacting to transient accuracy drops, each client's flag counter cnt_j tracks underperformance rounds. A client with trust score τ_j below $\tau_{thresh} = \frac{1}{kn}$ (a scalar multiple of n , adjustable via k to control detection speed) is flagged (Algorithm 2, lines 8–9). After cnt_{max} flags, a client is deemed malicious and added to list β (lines 10–11). Flagged clients ($\tau_j < \tau_{thresh}$) have updates excluded from the next round's aggregation to prevent model contamination. Malicious clients are excluded from participation from future rounds. Trust values are normalized to $[0, 1]$ (line 12). This balances tolerance for noisy data with protection against adversarial behavior.

The overall malicious node filtering process is implemented in Algorithm 2, which is invoked from the main AntiFLipper algorithm (Algorithm 1, line 6) during each communication round, before global model aggregation.

E. M5: Weighted Aggregation Module

After malicious clients have been filtered out, the remaining client models are aggregated using a weighted average (Algorithm 1, line 7). The weight assigned to each client's model is

Algorithm 2: MalNodeFilter()

Input : Clients $clients$, Accuracies acc_{ij} , Trust values τ , Flags cnt , Malicious list β

Output: Updated τ , cnt , β

```
1  $acc_{avg} \leftarrow \text{getAvgAccuracy}(clients \setminus \beta)$ 
2 foreach  $j \in clients$  and  $j \notin \beta$  do
3    $d \leftarrow \text{sqDeviation}(acc_{ij}, acc_{avg})$ 
4   if  $acc_{ij} < acc_{avg}$  then
5      $\tau_j \leftarrow \text{penalizeTrust}(\tau_j, d, \alpha)$ 
6   else
7      $\tau_j \leftarrow \text{rewardTrust}(\tau_j, d, \alpha)$ 
8   if  $\tau_j \leq \tau_{thresh}$  then
9      $cnt_j \leftarrow cnt_j + 1$ 
10    if  $cnt_j \geq cnt_{max}$  then
11      add  $j$  to  $\beta$ 
12  $\tau \leftarrow \text{normalizeTrust}(\tau, \beta)$ 
13 return  $\tau$ ,  $cnt$ ,  $\beta$ 
```

proportional to its normalized trust score. Formally, the aggregated model M_{i+1} is computed as: $M_{i+1} = \sum_{j \in \text{Honest}} \frac{\tau_j}{W} \cdot m_{ij}$, where $W = \sum_{j \in \text{Honest}} \tau_j$. This ensures that reliable clients have a greater impact on the evolution of the global model, reinforcing the positive feedback loop while suppressing the influence of detected adversaries.

V. IMPLEMENTATION AND EVALUATION

We developed AntiFlipper¹ using Python and evaluated it against label-flipping attacks, comparing it with established defenses including FedAvg [1], Median, Trimmed Mean [10], FoolsGold [6], Tolpegin [9], MKRUM [15], FLAME [11], and LFighter [14]. Experiments were conducted using PyTorch on a 12th Gen Intel Core i9-12900k processor with 128GB RAM and an NVIDIA RTX 3090 GPU, running Ubuntu 22.04.4 LTS. We used two benchmark datasets: MNIST70K[16], consisting of 70,000 grayscale handwritten digit images (0–9), split into 60,000 for training and 10,000 for testing; and CIFAR10[17], with 60,000 color images across 10 classes, split 50K/10K for training/testing. For MNIST, we used a two-layer CNN followed by two fully connected layers, while CIFAR10 models employed ResNet18 [18] and ShuffleNetV2 [19], each ending with one fully connected layer.

Experiments were run under both IID and non-IID data distributions. MNIST IID used random uniform splits among 100 clients, and non-IID used Dirichlet sampling with $\alpha = 1$. Models trained for 200 rounds with 3 local epochs, batch size 64, using cross-entropy loss and SGD (lr = 0.001, momentum = 0.9). For CIFAR10, IID involved 100 peers with 20 active per round, and non-IID used Dirichlet sampling among 20 peers ($\alpha = 1$). These models trained for 100 rounds with 3 local epochs, batch size 32, and SGD (lr=0.01, momentum=0.9). In all settings, we simulated 40% malicious clients, the theoretical

upper bound MKRUM [15] can tolerate. These adversaries launched constant label-flipping attacks, altering multiple labels in their local datasets.

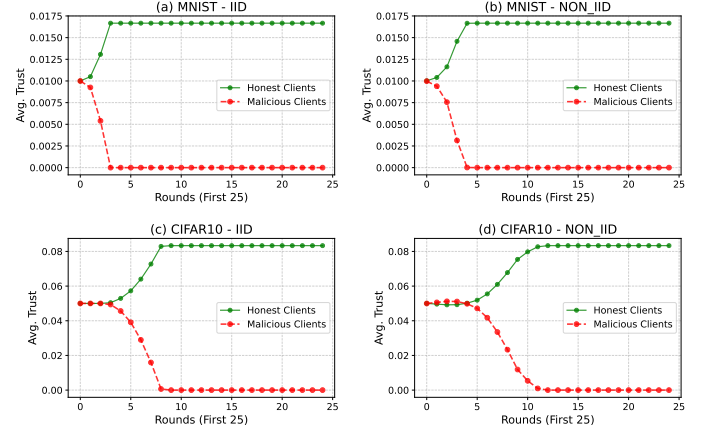


Fig. 2: Average Client Trust Over First 25 Rounds

A. Evaluation Metrics

We evaluated our approach using three metrics: Overall Accuracy (All-Acc), Test Error (TE), and Aggregation Time. All-Acc indicates prediction correctness, directly measuring model performance. TE, derived from cross-entropy loss, reflects how well predicted probabilities match true labels—critical for assessing robustness against label-flipping. Aggregation Time measures the efficiency of combining client updates, essential for scalability.

B. Malicious Node Detection

AntiFlipper distinguishes malicious from honest clients using the global model’s accuracy on their local datasets. When a client flips labels, its local accuracy drops because the global model, shaped by honest nodes, performs poorly on manipulated data. Malicious node accuracy declines rapidly, reaching near zero early on MNIST and within 20 rounds on CIFAR, while honest nodes maintain higher accuracy. Figure 2 presents average trust scores, computed from deviations between local and global accuracy, and clearly shows how honest and malicious nodes diverge after only a few rounds. We focus on the first 25 rounds, where AntiFlipper detects adversaries early, and this trend continues throughout training.

C. Experimental Results

We evaluated defenses against label-flipping attacks on MNIST (CNN) and CIFAR-10 (ResNet18) under both IID and non-IID settings (Table I). On MNIST, AntiFlipper, LFighter, Tolpegin, MKRUM, FLAME, TMean, and Median all achieved high accuracy. Median and TMean benefited from MNIST’s low variability but rely on honest-majority assumptions and degrade on complex data. MKRUM offered a good balance of accuracy and overhead via trusted subset selection, though it struggled with subtle attacks. FLAME was accurate but computationally intensive. LFighter and Tolpegin were similarly effective but

¹The source code of AntiFlipper is available at: <https://github.com/abidh8820/AntiFlipper>

TABLE I: Comparison of Different Methods on MNIST and CIFAR-10 (IID and Non-IID)

	Methods	FoolsGold	FedAvg	Median	TMean	FLAME	MKRUM	LFighter	Tolpegin	AntiFLipper
MNIST (IID)	Global Accuracy (%)	0.36	51.85	96.10	96.18	97.44	97.45	97.45	97.45	97.06
	Test Error	17.60	0.86	0.18	0.18	0.09	0.09	0.09	0.09	0.10
	Aggregation Time (ms)	9.36	4.61	12.03	12.86	45.61	15.55	119.65	160.94	3.01
MNIST (NON-IID)	Global Accuracy (%)	3.58	53.99	95.67	95.61	97.03	96.74	96.76	96.80	96.77
	Test Error	6.72	0.76	0.22	0.23	0.11	0.12	0.13	0.13	0.11
	Aggregation Time (ms)	8.65	4.44	13.09	11.73	45.66	13.70	119.37	168.88	2.96
CIFAR (IID)	Global Accuracy (%)	70.13	47.21	63.90	63.77	67.30	32.86	69.96	70.28	70.09
	Test Error	1.10	1.43	1.10	1.12	1.15	2.31	1.12	1.11	1.10
	Aggregation Time (ms)	53.87	50.76	660.39	641.66	1489.91	686.93	468.02	354.96	48.01
CIFAR (NON-IID)	Global Accuracy (%)	56.01	50.95	60.93	59.66	37.77	44.08	64.58	53.58	68.23
	Test Error	1.12	1.31	1.18	1.19	3.23	1.70	1.21	1.31	1.14
	Aggregation Time (ms)	53.16	49.75	712.13	678.38	1334.06	727.38	449.12	339.47	48.43

slow. FoolsGold failed in non-IID settings (3.58% accuracy), misidentifying skewed honest gradients as sybils, while FedAvg amplified poisoned updates. AntiFLipper matched or exceeded others in accuracy with the lowest aggregation time, using lightweight flip-pattern detection for optimal efficiency.

CIFAR-10 results followed similar patterns with generally lower accuracy due to higher complexity. In IID, AntiFLipper (70.09%) performed on par with LFighter, Tolpegin, FLAME, and FoolsGold, again achieving the lowest aggregation time. In non-IID, AntiFLipper led in both accuracy and efficiency. Median, TMean, and MKRUM struggled due to dispersed or variable updates. FLAME remained effective but slow. FoolsGold improved on CIFAR due to clearer gradient patterns, but still faltered in non-IID (56.01%). AntiFLipper consistently outperformed others by maintaining high accuracy and low overhead across datasets.

We further tested AntiFLipper under dynamic attack scenarios: (1) attacks starting after $n/2$ rounds (with n the total rounds), and (2) poisoning every n rounds ($n = 15$). As shown in Table II, AntiFLipper maintained accuracies within $\pm 0.5\%$ of baseline and detected all malicious nodes. Delayed or intermittent attacks were easier to detect, as the model, already dominated by benign updates, remained stable and deviations were more apparent. In both cases, AntiFLipper remained robust, efficient, and incurred similar aggregation time as before.

TABLE II: Global test accuracies (%) under dynamic attack scenarios

Scenario	MNIST		CIFAR-10	
	IID	Non-IID	IID	Non-IID
Scenario 1	96.93	97.03	72.66	70.62
Scenario 2	96.68	96.82	72.59	70.59

In summary, AntiFLipper delivers robust, efficient, and scalable defense across datasets, distributions, and dynamic attack conditions.

VI. SECURITY ANALYSIS

In this section, we formalize AntiFLipper’s trust-based defense against label-flipping attacks. Our approach quantifies deviations in local model accuracy to adjust client trust scores,

enabling the system to progressively isolate and exclude malicious participants. This establishes AntiFLipper as an efficient and robust defense for secure federated learning in adversarial federated learning environments.

A. Accuracy-Based Detection of Label Flipping Behavior

A key challenge in federated learning is mitigating label-flipping attacks, where malicious clients poison training by mislabeling data. AntiFLipper tackles this by exploiting the accuracy drop observed when the global model is trained on such poisoned data.

Lemma-1: *In AntiFLipper, if a client c_i flips labels in its local dataset d_i , converting it into a poisoned distribution d'_i , then the global model M yields lower accuracy on d'_i than on honest data, exposing adversarial behavior.*

Proof: Let $C = \{c_1, c_2, \dots, c_n\}$ be the set of clients participating in federated learning, and let M be the global model aggregated from local models of the majority of honest clients. If a client $c_i \in C$ performs label-flipping, its local data distribution becomes d'_i , which diverges from the honest distributions d_j , where $j \neq i$. The global model M minimizes loss over honest client distributions d_j , so when evaluated on a flipped distribution d'_i , it incurs higher loss: $L(M, d'_i) > L(M, d_j)$. Since accuracy decreases as loss increases, this implies: $\text{acc}(M, d'_i) < \text{acc}(M, d_j)$. To detect deviation, AntiFLipper compares each client’s accuracy to the average acc_{avg} . For a malicious client c_i , $\text{acc}_{\text{avg}} - \text{acc}(M, d'_i) > \text{acc}_{\text{avg}} - \text{acc}(M, d_j)$, and squaring amplifies this gap: $\Delta_i = (\text{acc}_{\text{avg}} - \text{acc}(M, d'_i))^2$. This deviation is used to reduce the client’s trust score, allowing the system to gradually identify and isolate clients that consistently behave maliciously.

B. Trust Score Convergence for Malicious Client Exclusion

In federated learning, malicious clients can persistently degrade the global model through label-flipping attacks while remaining undetected. AntiFLipper mitigates this by assigning trust scores based on local evaluation performance and excluding clients whose scores consistently fall below a defined threshold.

Lemma-2: *In AntiFLipper, a client c_i is considered malicious and excluded from training if its trust score satisfies*

$\tau_i < \tau_{\text{threshold}}$ for more than x rounds, indicating consistent behavior associated with label-flipping attacks.

Proof: Each client $c_i \in C$ starts with a trust value $\tau_i = \frac{1}{n}$, ensuring $\sum_{i=1}^n \tau_i = 1$. These values are used to weight local models in global aggregation as $M \leftarrow M + w_i \cdot m_i$, where $w_i = \frac{\tau_i}{W}$, $W = \sum \tau_i = 1$. After each round, trust is updated based on the squared deviation from the average accuracy: $\tau_i \leftarrow \tau_i \pm \alpha \cdot (\text{acc}_i - \text{acc}_{\text{avg}})^2$, and then normalized to maintain $\sum \tau_i = 1$. Since trust updates are applied independently per client, individual scores fluctuate while the total remains constant. A client is flagged if $\tau_i < \tau_{\text{threshold}} = 1/kn$, where $k > 1$, indicating its influence has dropped by a factor of k . Given $\tau_i \ll \tau_{\text{avg}} = 1/n$, its contribution becomes negligible. If this persists for x rounds, the client is considered malicious and excluded from further training.

VII. COMPUTATIONAL OVERHEAD OF ANTIFLIPPER

To quantify the computational overhead introduced by local evaluation in AntiFlipper, we consider a neural network model $f(x; w)$ with W parameters, trained on each client k with dataset D_k of n_k samples. Clients perform E local SGD epochs (batch size B), where the per-sample forward and backward pass costs are C_f and $C_b \approx 2C_f$, respectively. Thus, the training cost per client per round is $C_{\text{train}} \approx 3E \cdot n_k \cdot C_f$.

Local evaluation on a fraction α of the data adds $C_{\text{eval}} = \alpha \cdot n_k \cdot C_f$, leading to a total cost of $C_{\text{total}} = n_k \cdot (3E + \alpha) \cdot C_f$, and relative overhead $\frac{\alpha}{3E}$. In our experiments with $E = 3$, full evaluation ($\alpha = 1$) yields a worst-case overhead of 11.1%. The global accuracies reported in Table I correspond to this full local data evaluation setting.

To reduce cost, we performed evaluation on only 10% of local data ($\alpha = 0.1$), resulting in just 1.1% overhead. Despite this, the system achieves strong performance: 97.11% (MNIST IID), 96.79% (MNIST Non-IID), 74.81% (CIFAR-10 IID), and 68.18% (CIFAR-10 Non-IID), with early malicious client detection, no increase in aggregation time, and no noticeable drop in accuracy—sometimes even slight improvements.

These results demonstrate that local evaluation can be both computationally lightweight and effective. By adjusting α , AntiFlipper offers flexible trade-offs between overhead and detection sensitivity, making it well-suited for resource-constrained federated learning environments. This adaptability ensures robust defense without compromising training efficiency or model quality.

VIII. CONCLUSION

In this paper, we introduced AntiFlipper, a lightweight and effective defense mechanism for detecting and mitigating label-flipping attacks in federated learning. Unlike existing methods that trade off security for efficiency, AntiFlipper achieves a strong balance between both. Through extensive evaluations on IID and non-IID data distributions across two benchmark datasets, we show that AntiFlipper consistently matches or exceeds the accuracy of state-of-the-art defenses. Remarkably,

it achieves significantly lower aggregation time—even outperforming the baseline FedAvg in some cases. This optimal trade-off makes AntiFlipper particularly suitable for real-world federated learning systems that require both robustness against adversaries and practical efficiency, especially in privacy-sensitive and time-constrained environments.

REFERENCES

- [1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Artificial intelligence and statistics*. PMLR, 2017, pp. 1273–1282.
- [2] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, "Federated learning: Challenges, methods, and future directions," *IEEE signal processing magazine*, vol. 37, no. 3, pp. 50–60, 2020.
- [3] P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, M. Bennis, A. N. Bhagoji, K. Bonawitz, Z. Charles, G. Cormode, R. Cummings *et al.*, "Advances and open problems in federated learning," *Foundations and trends® in machine learning*, vol. 14, no. 1–2, pp. 1–210, 2021.
- [4] L. Lyu, H. Yu, and Q. Yang, "Threats to federated learning: A survey," *arXiv preprint arXiv:2003.02133*, 2020.
- [5] B. Biggio, B. Nelson, and P. Laskov, "Support vector machines under adversarial label noise," in *Asian conference on machine learning*. PMLR, 2011, pp. 97–112.
- [6] C. Fung, C. J. Yoon, and I. Beschastnikh, "The limitations of federated learning in sybil settings," in *23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2020)*, 2020, pp. 301–316.
- [7] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, "How to backdoor federated learning," in *International conference on artificial intelligence and statistics*. PMLR, 2020, pp. 2938–2948.
- [8] L. Muñoz-González, B. Biggio, A. Demontis, A. Paudice, V. Wongrasamee, E. C. Lupu, and F. Roli, "Towards poisoning of deep learning algorithms with back-gradient optimization," in *Proceedings of the 10th ACM workshop on artificial intelligence and security*, 2017, pp. 27–38.
- [9] V. Tolpegin, S. Truex, M. E. Gursoy, and L. Liu, "Data poisoning attacks against federated learning systems," in *Computer security—ESORICS 2020: 25th European symposium on research in computer security, ESORICS 2020, guildford, UK, September 14–18, 2020, proceedings, part i 25*. Springer, 2020, pp. 480–501.
- [10] D. Yin, Y. Chen, R. Kannan, and P. Bartlett, "Byzantine-robust distributed learning: Towards optimal statistical rates," in *International conference on machine learning*. Pmlr, 2018, pp. 5650–5659.
- [11] T. D. Nguyen, P. Rieger, R. De Viti, H. Chen, B. B. Brandenburg, H. Yalame, H. Möllering, H. Fereidooni, S. Marchal, M. Miettinen *et al.*, "{FLAME}: Taming backdoors in federated learning," in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 1415–1432.
- [12] H. Wang, K. Sreenivasan, S. Rajput, H. Vishwakarma, S. Agarwal, J.-y. Sohn, K. Lee, and D. Papailiopoulos, "Attack of the tails: Yes, you really can backdoor federated learning," *Advances in neural information processing systems*, vol. 33, pp. 16 070–16 084, 2020.
- [13] X. Cao, M. Fang, J. Liu, and N. Z. Gong, "Fltrust: Byzantine-robust federated learning via trust bootstrapping," *arXiv preprint arXiv:2012.13995*, 2020.
- [14] N. M. Jebreel, J. Domingo-Ferrer, D. Sánchez, and A. Blanco-Justicia, "Lfighter: Defending against the label-flipping attack in federated learning," *Neural Networks*, vol. 170, pp. 111–126, 2024.
- [15] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," *Advances in neural information processing systems*, vol. 30, 2017.
- [16] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [17] A. Krizhevsky, G. Hinton *et al.*, "Learning multiple layers of features from tiny images," 2009.
- [18] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [19] N. Ma, X. Zhang, H.-T. Zheng, and J. Sun, "Shufflenet v2: Practical guidelines for efficient cnn architecture design," in *Proceedings of the European conference on computer vision (ECCV)*, 2018, pp. 116–131.